REDPANDA FDE EXERCISE

Wikipedia edit triage

From public events to grounded, inspectable decisions

Architecture, phased delivery, tests, failure modes and presentation preparation

STUDY GUIDE / 34 CHAPTERS

Prepared for Damien Gallagher • 20 October 2026

Based on repository snapshot 859b166. Includes PR #13 evidence, which was still under review when this pack was prepared.

Contents

A field guide to the implementation, the evidence and the questions it invites.

How to use this study pack	3	Phase 6 · Operational acceptance and handover	20
What the exercise actually asked for	4	How the evidence was assembled	21
The complete system	5	Local versus frontier: one contract, explicit choices	22
Why the work was divided into phases	6	What the final real demonstrations showed	23
Phase 0 · Make the work reviewable	7	Docker lifecycle, cleanup and persistence	24
Phase 1 · Foundation and model feasibility	8	Failure map: where does this input end up?	25
Foundation verification and model choice	9	A clean presentation demo, step by step	26
Phase 2 · From SSE frames to trusted events	10	Prove the end-to-end result on screen	27
Ingestion filters, tests and source limitations	11	The code tour: follow one input, not every file	28
Phase 3 · Fetch the evidence the model needs	12	Prompts, responses and evidence provenance	29
Enrichment bounds and explicit failure outcomes	13	Prioritized improvement roadmap	30
Phase 4 · Bounded multi-step reasoning	14	What must change for a real customer deployment	31
Trace a two-step result	15	Questions you should be able to answer	32
Dirty output, uncertainty and call budgets	16	A 60-minute walkthrough plan	33
Kafka delivery: acknowledge output, then commit input	17	Preparing for the pre-walkthrough extension	34
Phase 5 · Durable, idempotent persistence	18	Glossary and quick distinctions	35
Serving: queries, responses and proof of persistence	19	Source map and evidence register	36

01 / STUDY CHAPTER

How to use this study pack

Start with the story, then trace the evidence.

This project turns selected Wikipedia revision events into evidence-backed suggestions for human review. Redpanda Connect handles transport and transformation. A small Python worker fetches the actual revision diff, performs bounded multi-step reasoning, and publishes a versioned result. A second Connect pipeline stores that result in Postgres; a read-only API makes it inspectable.

Study in three passes. First, read the architecture and phase map to understand why each layer exists. Next, trace the worked examples and failure paths with the source files open. Finally, use the rehearsal questions and HTML presentation mode to practise explaining the design without reading a script.

Evidence label	How to interpret it
Implemented	Confirmed in the source snapshot, including the PR #13 follow-up.
Recorded observation	A saved run on a named commit; not a promise about future live inputs.
Synthetic example	A controlled fixture, mock provider response or labelled teaching projection.
Proposed improvement	Future engineering work; not a feature of the submitted application.

> **Version and review boundary**
> Source snapshot: 859b166, 20 October 2026. Main was 447920f; PRs #1-12 were merged. PR #13 was open at preparation time and contains the demo evidence plus helper-volume fix. The 30-integration-test total includes that fix; main before it has 29.

This is an AI-assisted study aid, not the assignment's independent personal write-up. The reserved README sections remain untouched. Use the evidence to form and write your own judgments. No new model calls or application executions are claimed by creation of this guide.

Source trail
README.md · docs/handover.md · docs/evidence/submission-demo-verification.json

02 / STUDY CHAPTER

What the exercise actually asked for

Translate the brief into observable contracts.

Requirement	Delivered component	How to demonstrate it
Pull a public source	Wikimedia recent-change SSE	Live ingestion record plus local HTTP fixture.
Transform in Connect	Framing, filters, projection, deterministic selection, routing	Native Bloblang tests and exact topic payloads.
Reason on actual content	MediaWiki comparison plus Python extraction / assessment / review	Show before/after evidence and two valid steps.
More than one prompt	Bounded branching and repair in Python	Trace reasoning.py and an uncertainty test.
Serve results	Connect SQL sink, Postgres, FastAPI	Read an existing analysis with full provenance.
Test risky logic	Strict parser, grounding, budgets, offsets and replay	Deliberate mutations fail targeted tests.
Run locally	Docker Compose and private mode profiles	Fresh startup, readiness, batch completion and cleanup.

Additional requirements from Damien

The user added reviewable phases and PRs, production-quality engineering practices, 100% application line and branch unit coverage, exact verification instructions, selectable local/frontier models, and a visible prompt/response audit trail. Those are requirements beyond the assignment's minimum focused test.

Resolve the brief's ambiguous hint

The brief includes a generic Connect branch hint for LLM calls, but explicitly requires the reasoning loop to live in application code. This implementation follows the explicit requirement: Connect does plumbing; Python owns model calls, recovery and uncertainty decisions. No Redpanda Cloud, enterprise-only connector or a2a_message is used.

> **A useful distinction**
> Production-quality boundaries and tests are implemented. A single-node local deployment is not a certified production platform. Coverage, model quality and operational readiness are three separate claims.

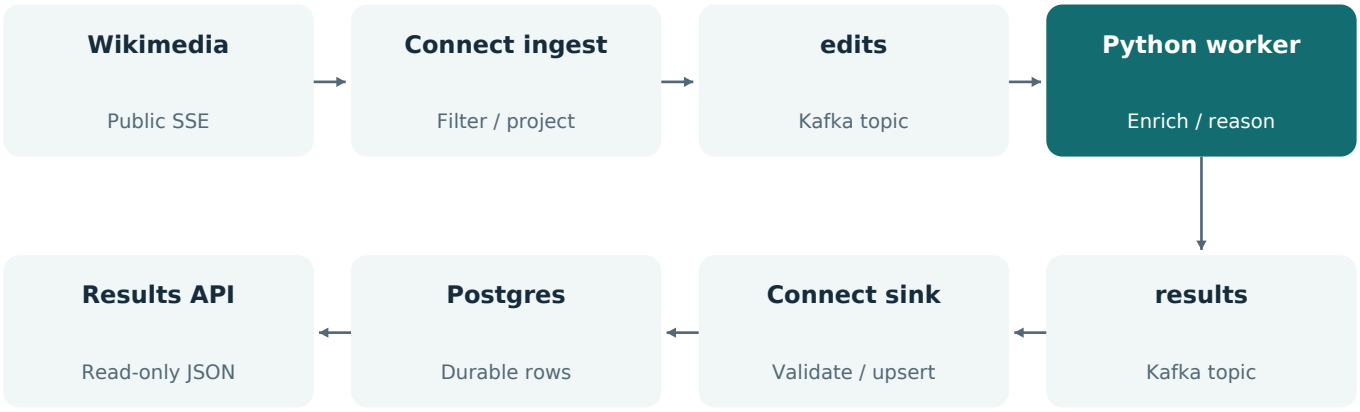
Source trail
docs/assignment/fde-take-home-exercise.md · docs/implementation-plan.md · docs/handover.md

03 / STUDY CHAPTER

The complete system

Three independent concerns: move data, reason on evidence, serve results.

Docker Compose runs the local services. Model: Ollama OR hosted OpenAI.



Side routes: ingest-errors | processing-errors | sink-errors. MediaWiki supplies the revision diff.

Component	Its responsibility	What it deliberately does not own
Redpanda broker	Kafka-compatible topics, offsets, retention and decoupling	Semantic classification or a user-facing results database.
Connect: ingest	Read SSE; filter, validate, project and route	Model orchestration or confidence decisions.
Python worker	Fetch diff, validate provenance, reason, publish then commit	SQL writes or synchronous API inference.
Connect: sink	Validate results; parameterized SQL upsert; quarantine invalid records	Re-run the model or decide a triage label.
Postgres + API	Durable queryable results, full evidence, pagination	Automatic edits to Wikipedia or truth verification.
Docker Compose	Local service dependencies, networking and volumes	High availability or production orchestration.

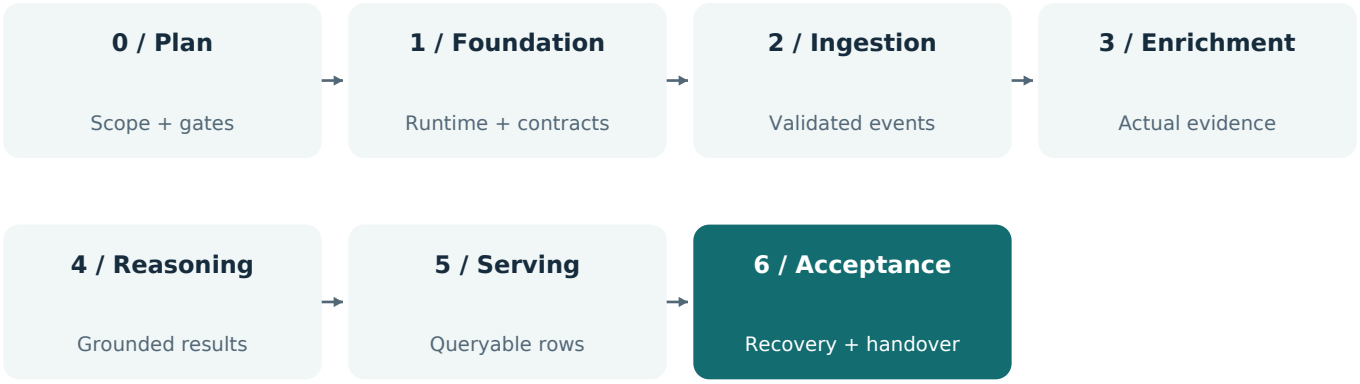
Ollama and the hosted provider occupy the same model boundary. They are explicit alternatives selected by configuration, not a fallback chain. The API reads persisted data and never triggers inference. This separation lets storage recover independently and allows results to be replayed without paying for the model again.

Source trail
compose.infrastructure.yml · docker-compose.yml · src/triage/worker.py · connect/sink.yaml

04 / STUDY CHAPTER

Why the work was divided into phases

Each phase creates a contract the next phase can trust.



Every phase adds an observable output and preserves earlier contracts through cumulative checks.

Phase	Observable output	Next phase depends on
0 · Plan	Scope, contracts, tests and review gates	A shared definition of completion.
1 · Foundation	Working broker/database/model interfaces	Stable infrastructure and typed events.
2 · Ingestion	Validated EditEvent records in edits	Correct revision identity, keys and timestamps.
3 · Enrichment	Bounded before/after evidence with provenance	Actual content to reason about.
4 · Reasoning	Versioned results or technical error records	Acknowledged, inspectable analysis output.
5 · Serving	Idempotent SQL rows and read-only queries	A complete source-to-consumer path.
6 · Acceptance	Recovery, clean startup, evaluation and handover	Evidence that the assembled system behaves as documented.

This order kept failures attributable. Ingestion could be verified without a model. Enrichment could be inspected without Kafka. Reasoning could use controlled HTTP responses while exercising a real broker. Serving could replay synthetic results before real inference was introduced.

Review sequence: planning PR #1 and assignment/test documentation #2; foundation #3 with follow-ups #4-5; ingestion and launchers #6; enrichment #7; reasoning #8; serving #9 and handover #10; acceptance/CI #11; walkthrough #12; executed-demo fixes/evidence #13. Damien reviewed and merged phases; the agent did not auto-merge.

Source trail
docs/progress.md · docs/phases/README.md · docs/implementation-plan.md

05 / STUDY CHAPTER

Phase 0 · Make the work reviewable

Purpose: agree the product and define evidence before implementation.

What was built

The plan selected Wikipedia edit triage, Python, explicit local/hosted inference, and a fresh build. It documented a six-phase dependency chain, phase-level commands, quality gates, the original assignment, and the visible interaction log. The initial history reset was explicitly requested; subsequent phase history was preserved.

Inputs and outputs

Input	Output
Assignment + user preferences	A small content-based review system with no automatic reverts.
Unknown reviewer hardware	A local path and a hosted path behind one interface.
Need to work intermittently	Progress notes, tested commits, next steps and branch/PR boundaries.
Need to verify each increment	Per-phase commands, expected output, failure cases and retained evidence.

How it was checked

Planning checks verified document links, the assignment's byte-for-byte copy/checksum, branch policy and test descriptions. There were no application tests or inference results to claim at this phase. Proposed commands were marked as future interfaces until implemented.

What the next phase could rely on

Phase 1 inherited an agreed architecture and explicit acceptance criteria, not working software. The key planning decision was to fetch real revision content before reasoning: an edit comment or bot flag alone would allow a simple rule to replace the model.

> **Rehearsal prompt**
> Explain the difference between a plan, a passing test and an observed live result. Point to one artifact for each; do not treat a planned command as evidence of implementation.

Source trail
docs/phases/00-plan.md · docs/implementation-plan.md · docs/phase-verification.md · docs/ai-log/README.md

06 / STUDY CHAPTER

Phase 1 · Foundation and model feasibility

Purpose: prove the runtime and provider boundaries before adding the stream.

Built single-node Redpanda, Postgres, persistent volumes, health checks, idempotent topic/schema initialization, a typed Python package, configuration validation, a non-root container, locked dependencies and CI. Strict event/result contracts reject extra fields, bad types, non-finite confidence and inconsistent evidence.

The model adapter performs one bounded request to the selected backend. The worker introduced later owns retries. This avoids hidden retries multiplying the overall call budget. No silent provider fallback is allowed: switching to hosted inference must be a deliberate configuration choice.

Boundary input	Observed or required output
Valid local settings	Ollama /api/chat with the selected model.
Valid hosted settings + private key	OpenAI /chat/completions; no local model dependency.
LLM_BACKEND=invalid	Configuration error, exit 2, no provider call.
Repeated topic/SQL initialization	Existing topics and one migration marker remain valid.
Broker round-trip fixture	Exactly the produced payload is consumed.

A real smoke input, identical across providers

```
Before: The study found a possible association, but did not establish causation.
After: The study proved that the treatment causes recovery.
```

Exact synthetic Phase 1 smoke input

Ollama replied: "The meaning changed from suggesting a possible link to asserting a definitive causal relationship." The hosted reply described the same shift from tentative association to definitive causation. These prove connectivity and basic feasibility; they are not model-accuracy measurements.

> **Phase boundary**
> No live ingestion, multi-step worker, sink or results API existed yet. Phase 2 could now rely on the broker, schemas and reproducible packaging.

Source trail
docs/phases/01-foundation.md · src/triage/config.py · src/triage/llm.py · docs/evidence/phase-1-local-model.json · docs/evidence/phase-1-hosted-model.json

07 / STUDY CHAPTER

Foundation verification and model choice

Purpose: establish a reproducible baseline, not just healthy containers.

Check	What it proves
86 original unit tests	Config, schemas, adapters and CLI behavior; 176 lines / 32 branches covered.
3 initial infrastructure checks	Hosted configuration, real broker round trip and idempotent persistent SQL.
Postgres maintenance follow-up	16.14 to 16.15 retains a sentinel row/migration timestamp and accepts writes; integration total becomes 4.
Wheel/container check	The packaged CLI runs as non-root user 10001.
Real provider smoke	Qwen2.5 7B: 7.788 s; hosted GPT-6.1 Sol: 2.911 s for one synthetic request.

Why Qwen2.5 7B became the local default

The recorded Qwen3 4B candidate took 25.687 seconds and produced an unwanted long explanation. Qwen2.5 7B returned the concise semantic comparison the smoke requested. This is a documented feasibility choice on one machine, not a general ranking of the two model families.

Resource and reproducibility boundaries

Recorded hardware was an ARM64 Mac with 32 GiB RAM. The default local weights are about 4.7 GB. Documentation suggests roughly 12 GiB Docker RAM and 12 GB free disk for local mode, or 4 GiB Docker RAM for hosted mode; those are starting allocations, not benchmarked minimums.

Container digests and Python dependencies are pinned. Current snapshot: Python 3.13.16, uv 0.13.4, Redpanda 26.2.5, Connect 4.114.0, Postgres 16.15 and Ollama 0.40.6. These are recorded project pins, not a claim about today's latest releases. Model tags remain mutable, so evaluation evidence records the actual model identity/digest.

```
make verify-phase-1
make test-unit
make test-container
```

Current commands; historical counts belong to their recorded commits

> **What to be ready to explain**
> Why the adapter rejects truncated responses; why provider failures are sanitized; why the same model interface does not imply every hosted provider is compatible.

Source trail
docs/phases/01-foundation.md · docs/phases/01-dependency-updates.md · docs/phases/06-ci-maintenance.md · pyproject.toml

08 / STUDY CHAPTER

Phase 2 · From SSE frames to trusted events

Purpose: remove noise and establish a compact transport contract.

Connect reads the public stream, normalizes supported LF/CRLF event-first framing, joins data lines, ignores heartbeats, validates candidates, filters irrelevant changes and selects a deterministic subset. It then projects only the fields needed to retrieve and identify a revision. Raw editor/IP metadata is not carried into the event contract.

```
event: message
id: fixture-1
data: {"wiki":"enwiki","type":"edit","namespace":0,
       "bot":false,"title":"Example article",
       "revision":{"old":100,"new":101},"timestamp":1792490400}
```

Teaching projection of a synthetic SSE frame; actual fixture also tests framing boundaries

```
{
  "schema_version": 1,
  "wiki": "enwiki",
  "title": "Example article",
  "old_revision_id": 100,
  "revision_id": 101,
  "occurred_at": "2026-10-20T10:00:00Z"
}
```

First expected fixture record; see source fixture for the exact broker envelope

Accepted records use key enwiki:101 and headers source=fixture, source_id=fixture-1, ingest_version=1. Stable identity is essential for replay inspection and downstream logical deduplication.

Two outcomes that should not be confused

• Intentional exclusion: a bot, non-article, non-English or unselected edit produces no record in edits. This is filtering, not a processing error.

• Malformed candidate: a bounded error envelope goes to ingest-errors with a reason, source ID and payload hash. It omits raw malformed content.

> **Next dependency**
> Phase 3 receives validated old/new revision IDs and direction. It still has to fetch the actual changed content; metadata alone cannot establish a semantic change.

Source trail
connect/ingest.yaml · connect/ingest_test.yaml · tests/fixtures/ingestion/expected-edits.json · docs/phases/02-ingestion.md

09 / STUDY CHAPTER

Ingestion filters, tests and source limitations

Purpose: distinguish source selection from delivery guarantees.

Rule or bound	Why it matters
enwiki + edit + namespace 0 + bot=false	Drops irrelevant events before any expensive enrichment or model work.
new_revision_id % 100 == 0	Default deterministic 1% selection of eligible ID space; not a strict rate limit.
INGEST_SAMPLE_ALL=true	Demo override; can create substantial backlog even with a three-item worker.
Positive, distinct integral IDs below 2^53	Avoids corrupted identity from numeric coercion or formatting.
64 KiB supported frame; 128 KiB scanner cap	Processor quarantine and upstream scanner failure are separate boundaries.

Meaningful test evidence

Forty native Connect cases exercise framing, each filter, selection and field bounds. A real local HTTP source feeds the pinned Connect image and real broker. One fixture pass produces exactly two edits and one error; replaying LF and CRLF variants verifies stable keys and exact payloads. Tests inspect complete topic contents, so both missing and extra records fail.

A live run exposed large revision keys formatted in scientific notation. Explicit decimal formatting fixed it; regression cases include 1381188467 and 9007199254740991. Removing the bot filter in an isolated mutation made the native filter test fail. These checks show assertions that can detect an actual defect.

Recorded live observation

The Phase 2 metrics snapshot counted 789 processed segments: 25 accepted, 763 filtered and one heartbeat. After graceful draining, the broker contained 27 valid edits and no errors. The counts differ because the snapshot precedes shutdown; processing counters are not acknowledged-output counts.

> **Guarantee boundary**
> The implementation supports Wikimedia's observed event-first SSE profile, not arbitrary SSE ordering. Reconnect has no durable Last-Event-ID checkpoint. Events may be missed or repeated upstream; Kafka guarantees begin after broker acknowledgement.

Source trail
docs/phases/02-ingestion.md · docs/evidence/phase-2-live.json · tests/test_ingestion.py

10 / STUDY CHAPTER

Phase 3 · Fetch the evidence the model needs

Purpose: replace metadata-only labeling with content-based analysis.

The enrichment client sends validated revision IDs to a fixed HTTPS English Wikipedia Compare endpoint, with explicit from/to direction, a descriptive User-Agent, no redirects and bounded I/O. It verifies the returned revision IDs, article namespace and page identity before accepting the diff.

The HTML table diff is parsed into plain wikitext hunks. Direction, surrounding context, spaces, entities and citation markup are preserved. A ready result includes provenance and exact before/after evidence. It does not mean the full article or cited sources were fetched.

Actual attributed fixture

The Belle (Disney character) comparison uses revisions 1380845740 to 1381189012, page ID 4483446. The before text names Paige O'Hara, Susan Egan and Julie Nathanson; after retains Paige O'Hara. This establishes a content removal. It does not establish which revision is factually correct.

```
Input: EditEvent(old_revision_id=1380845740, revision_id=1381189012)
Output: status="ready", source="fixture", attempts=0
        provenance={endpoint, page_id, from_title, to_title, source_url}
        evidence=[{before: "...three names...", after: "...one name..."}]
        content_truncated=false
```

Abbreviated teaching projection; ellipses are not stored evidence

Inspect it without Kafka or a model

```
make inspect-enrichment
make test-enrichment
make smoke-enrichment
```

The smoke performs public GET requests; the other two use saved inputs

The recorded live comparison matched the fixture exactly in 0.287 seconds, with no model call. The deterministic gate had 214 unit tests, 10 integration/configuration checks and 40 Connect cases; all 451 application lines and 142 branches were covered.

> **Next dependency**
> Phase 4 can reason over a validated source result, and can route missing or incomplete evidence explicitly instead of manufacturing a confident label.

Source trail
src/triage/enrichment.py · src/triage/diff.py · docs/phases/03-enrichment.md · docs/evidence/phase-3-live.json

11 / STUDY CHAPTER

Enrichment bounds and explicit failure outcomes

Purpose: prevent oversized, mismatched or unsupported data becoming "complete evidence".

Boundary	Implemented behavior
Response bytes	Stream at most 256 KiB; reject unexpected compression and close on overflow.
Evidence	At most 20 hunks; 4,096 characters per side; 16,000 characters total. Whole over-limit hunks are omitted and incompleteness is explicit.
Time and retries	20-second cooperative budget, each I/O wait at most 5 seconds; at most two serial requests for retryable failures.
Retry-After	Honor valid values only within the 10-second retry and remaining-time limits; do not retry early.
Identity / structure	Validate revision/page/namespace identity and supported table structure; parser nesting limit 16.

Example	CLI result	Worker interpretation
Missing/suppressed revision	revision_unavailable; exit 1	insufficient_evidence with zero model calls.
Oversized evidence	evidence_limit; truncated=true; exit 1	Abstain; never routine on incomplete content.
Valid empty comparison	no_text_change; exit 0	Explicit no_change abstention, zero calls.
Unsupported markup or network failure	Explicit status; exit 1	enrichment_failed technical record.
Invalid input file/schema	Exit 2	Rejected before enrichment.

Tests check provenance mismatch, insertions/deletions, escaped markup, retry timing and exact evidence. Reversing before/after made the evidence test fail. A slow-stream regression caught an implementation that buffered all 100 simulated chunks before checking time; the fix stops at chunk five and closes the stream.

> **Important nuance**
> A cooperative deadline is checked between operations; it is not hard cancellation. An in-flight bounded I/O wait may exceed the nominal budget. More context or citation retrieval is future work, not a hidden feature of this client.

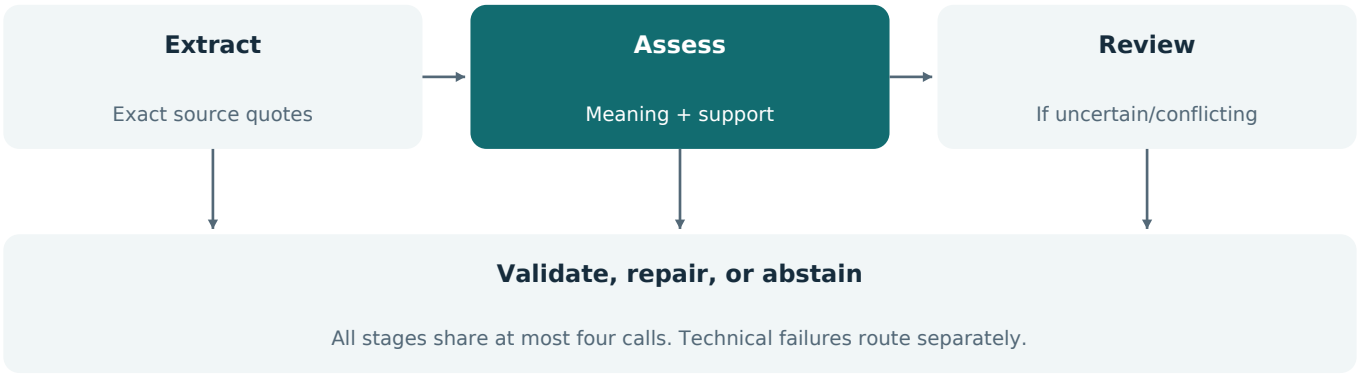
Source trail
docs/phases/03-enrichment.md · src/triage/enrichment.py · tests/test_enrichment.py

12 / STUDY CHAPTER

Phase 4 · Bounded multi-step reasoning

Purpose: give the model semantic work while keeping control flow in code.

Unavailable/empty/incomplete supported evidence bypasses model calls.



Extraction identifies changes in meaning and support, quoting both sides of the same hunk. Assessment compares that extraction against the original evidence and chooses routine, review or insufficient_evidence. These are separate model tasks; neither edits Wikipedia nor asserts factual truth.

A routine result conflicts with extraction when any change kind is not wording. That conflict, or confidence below 0.8, requests a second assessment. The review must still be well formed, sufficiently confident and consistent. Unresolved conflict or uncertainty produces abstention. A model-supplied confidence is a routing signal, not a calibrated probability.

Each stage gets at most two attempts, including repair or technical retry, and all stages share four model calls. A valid insufficient_evidence assessment can end immediately. Non-ready evidence that the worker treats as safely unavailable bypasses inference completely.

> **Why this is more than classification by metadata**
> The model compares actual changed text, explains changes to qualifiers, claims, entities and citations, and grounds its output in source quotes. Deterministic code validates and routes; it does not replace the semantic comparison with an edit-comment keyword rule.

Source trail
src/triage/reasoning.py · src/triage/prompts.py · docs/phases/04-reasoning.md

13 / STUDY CHAPTER

Trace a two-step result

A controlled example makes the contract easier to defend.

The deterministic worker demonstration changes an association without proof of causation into a statement that treatment causes recovery. The service is real; the Wikipedia and model HTTP responses are controlled test doubles. The exact model response is not being claimed as a new live provider run.

```
{"changes":[{"hunk":0,"kind":"meaning",
  "summary":"A qualified association becomes a causal claim.",
  "before":"The study found an association, without proving causation.",
  "after":"The study proved that the treatment causes recovery."}]}
```

Illustrative extraction shape; exact test text and full transcripts are linked below

Validation requires all hunks to be represented and each non-empty quote to be an exact contiguous excerpt from the corresponding side of that same hunk. Invented quotes, quoting the wrong hunk, or omitting a hunk fail before assessment.

```
{"decision":"review","confidence":0.95,
 "explanation":"The edit strengthens an association into a causal claim."}
```

Illustrative assessment; confidence is not a factual-truth score

Result boundary

```
Kafka key: enwiki:101:triage-v1
analysis: decision=review, attempts=2, backend/model/prompt_version
enrichment: original event, provenance, before/after evidence
extraction: grounded changes
steps: extract/valid -> assess/valid
outcome: assessed
```

Observed shape of the valid deterministic worker scenario

The prompts explicitly treat titles, article text and prior model output as untrusted data. Runtime logs retain stage/decision/offset metadata, not routine full prompts. Curated evaluations separately preserve visible inputs and outputs for review. No hidden chain-of-thought is recorded.

> **Two versions, two jobs**
> edit-triage-v2 identifies prompt templates. triage-v1 identifies the analysis/storage namespace. They are not interchangeable, and changing the model does not automatically create a new storage key.

Source trail
src/triage/prompts.py · src/triage/reasoning.py · tests/fixtures/worker/scenario.py · docs/evidence/phase-4-delivery.json

14 / STUDY CHAPTER

Dirty output, uncertainty and call budgets

Purpose: fail explicitly instead of accepting plausible-looking garbage.

Condition	Response
Bare JSON or exact JSON fence	Parse one object, then apply strict schema and semantic validation.
Prose around JSON, multiple objects, duplicate keys	Reject and make one fresh repair attempt for that stage, if budget remains.
Wrong enum/type, extra fields, non-finite confidence	Reject; never silently coerce an invalid label into routine.
Fabricated or incomplete source quotes	Reject the extraction before its claims can drive a result.
Repeated invalid output / unresolved review	Persist insufficient_evidence with an explicit outcome.
Exhausted model transport retry or deadline	Publish processing-errors, not a semantic result.

Repair sends the source/schema again with repair=true. It does not feed a detailed validator error or the rejected response back into the extraction prompt. Review receives the extraction and first assessment, both treated as untrusted data. This distinction matters when explaining what the model actually sees.

Resource budgets

The default completion budget is 1,024 tokens per call; total prompt text is capped at 32,000 characters, provider response bytes at 64 KiB, and input events at 8 KiB. Extraction permits up to eight changes with 512-character quotes. The source parser can accept 20 hunks, but extraction must cover every one: a larger number can therefore exhaust validation and abstain. This is an explicit improvement opportunity.

The worker starts a shared 240-second cooperative item deadline before enrichment. Network timeouts are reduced to remaining time. Slowly progressing I/O can still exceed the nominal wall-clock deadline before control returns; do not describe this as hard real-time cancellation.

> **Evidence of test strength**
> Removing duplicate-key rejection or lowering the confidence threshold broke targeted tests. The deliberately red demonstration also fails when it wrongly expects an invented quote to be accepted. The normal suite stays green.

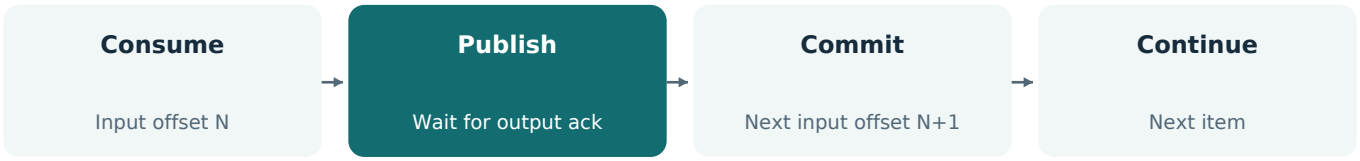
Source trail
src/triage/reasoning.py · src/triage/llm.py · docs/evidence/phase-4-mutations.json · tests/fixtures/acceptance/intentional_failure.py

15 / STUDY CHAPTER

Kafka delivery: acknowledge output, then commit input

Purpose: avoid losing accepted input when downstream publication fails.

Manual offset control; no transaction spans inference, Kafka and Postgres.



Crash after publish, before commit

Output may exist while input replays: duplicate processing is possible.

The worker consumes sequentially. Auto-commit and automatic offset storage are disabled. It publishes a result or technical error, waits for the producer delivery acknowledgement, then synchronously commits the next input offset. Only then does it fetch/process another item.

Failure point	What remains	Expected recovery
Before output acknowledgement	Input uncommitted; no durable output guarantee	Stop and replay from the committed position.
After output ack, before input commit	Output may exist; input is still pending	Replay can repeat inference and publish a duplicate.
After input commit	Input accounted for by acknowledged output	Continue from the next offset.
SIGTERM during inference	Stop flag set; current record may finish	Drain current item, commit it, leave next input pending.

A real missing-result-topic test keeps the committed offset at -1001 (the client's "not set" value). After topic creation and restart, two pending inputs yield two outputs and committed offset 2. A failure after publication produces three Kafka records for two inputs; Phase 5 collapses the duplicate into one logical SQL row.

> **Do not claim exactly once**
> Producer idempotence handles producer-level retries; it does not deduplicate new inference after process restart. Kafka and Postgres are not joined by one distributed transaction. Also, acks=all acknowledges only one replica in this single-node setup.

Source trail
src/triage/broker.py · src/triage/worker.py · docs/evidence/phase-4-delivery.json · docs/evidence/phase-6-crash.json

16 / STUDY CHAPTER

Phase 5 · Durable, idempotent persistence

Purpose: make results queryable without coupling the worker to SQL.

A second Connect pipeline consumes results, validates its JSON schema and cross-field invariants, checks the analysis key and provenance, then performs a parameterized upsert. Invalid records go to sink-errors; database outages remain retryable delivery problems. The worker still only publishes Kafka messages.

```
UNIQUE (wiki, revision_id, analysis_version)

INSERT ... ON CONFLICT (...) DO UPDATE ...
WHERE existing.source_offset < incoming.source_offset
```

Conceptual projection of the actual parameterized upsert

The unique key provides one logical row per analysis identity. A newer result-topic offset can update the payload, preserving row ID and creation timestamp. Older or equal offsets cannot overwrite that row during replay. This protects against replaying old Kafka records, not against a newly produced higher-offset result that contains different model output.

Migration and API boundaries

Migration 2 creates the result table and indexes with transactional, idempotent DDL and an advisory lock. It runs on startup before sink/API readiness, including against older Phase 1 volumes. No database reset is required for that migration.

The Python storage layer uses parameterized read-only queries with connection/query/lock timeouts. It validates stored payloads before serving them. The API exposes summaries and complete details, not raw SQL or untrusted HTML. It makes no model requests.

> **Single-partition assumption**
> The source-offset replay guard assumes results partition 0 and stable topic history. Other partitions are quarantined. Adding partitions or recreating a topic while retaining the database needs a new checkpoint/version design; it is not a safe one-line scaling change.

This is a replayable architecture: a new sink consumer group can rebuild SQL from retained results without rerunning inference. Replaying edits through the worker is different and can incur new model calls.

Source trail
connect/sink.yaml · db/002-results.sql · src/triage/storage.py · docs/phases/05-serving.md

17 / STUDY CHAPTER

Serving: queries, responses and proof of persistence

Purpose: let a reviewer inspect an analysis and its evidence.

Request	Expected behavior
GET /health/live	200 alive independent of database availability.
GET /health/ready	200 ready only if migration 2 and result table are readable.
GET /results?analysis_version=triage-v1&limit=10	Summaries in descending storage-ID order, with next_before_id.
GET /results/enwiki/101?analysis_version=demo-v1	Full synthetic stored result or a sanitized 404.
Invalid query / unavailable database	422 invalid_request / 503 storage_unavailable.
GET /docs	Generated interactive API documentation.

List limit is 1-100 (default 20). The analysis-version filter is optional on the list and required on detail. Use the returned before_id cursor with the same filters. Updates keep their ID/position. Paging is not a multi-request snapshot, so concurrent changes can be observed.

A safe demonstration without a model

```
make seed-results
curl --fail 'http://localhost:8000/results?analysis_version=demo-v1&limit=2'
curl --fail 'http://localhost:8000/results/enwiki/101?analysis_version=demo-v1'
```

Uses selected demo stack; adjust the port if API_PORT differs

The seed publishes three explicitly synthetic decisions: 101 review, 102 routine, 103 insufficient_evidence, with model fixture-not-a-real-model and version demo-v1. Repeating it leaves three logical rows. These IDs are not evidence of real Wikipedia judgments.

The Phase 5 gate passed 371 unit, 25 integration and 72 native Connect cases. A real database outage held the sink commit at 6; recovery advanced it to 7 and stored the pending analysis. Mutating pagination from < to <= made page two repeat revision 102 and failed the test.

> **Verify delivery, not just health**
> A ready API does not prove the sink caught up. Check triage-store-v1 lag and query the expected analysis key. List responses are summaries; retrieve detail to inspect original evidence and reasoning-stage outcomes.

Source trail
src/triage/api.py · src/triage/storage.py · docs/evidence/phase-5-serving.json · tests/test_serving_integration.py

18 / STUDY CHAPTER

Phase 6 · Operational acceptance and handover

Purpose: prove the assembled system can start, fail safely and be operated.

Added a small test/cleanup menu, separate local/frontier model tests, a frozen acceptance set, real broker outage/process-death recovery, safe data/model cleanup, a fresh local startup check, security review and handover documentation. These exercises test behavior beyond a successful happy-path request.

Command	What it demonstrates
make test	Cumulative deterministic gate; no live model/key required.
make demo-success	Controlled source through real Connect, broker, worker, Postgres and API.
make demo-failures	Nineteen selected scenarios covering faults and recovery; PASS means expected handling worked.
make demo-test-failure	One deliberately wrong assertion fails; Make exits 2.
./scripts/test-local.sh	Real local model, nine frozen cases, saved visible transcripts.
./scripts/test-frontier.sh	Real hosted model, same cases, up to 28 paid calls.

Observed improvements during acceptance

The image was hardened using a two-stage build and pinned Debian 13 Python base with runtime build tools removed. CI caught a floating-point clock assertion and it was fixed with a deterministic clock. Later action/dependency maintenance moved actions to native Node 24, adopted the supported test transport and made Python warnings errors.

The executed follow-up found CLI helpers inheriting anonymous image volumes. Giving only topics and db-migrate small temporary filesystems fixed the leak. A real-Docker regression was red before the fix and green after it; the latest integration total is 30.

> **Scope of acceptance**
> The tests establish specific local startup, delivery, recovery and cleanup contracts. They do not establish high availability, calibrated model accuracy, a security certification or production load capacity.

Source trail
docs/phases/06-acceptance.md · docs/phases/06-ci-maintenance.md · docs/evaluations/submission-demo.md

19 / STUDY CHAPTER

How the evidence was assembled

100% unit coverage is one layer of a larger verification strategy.

Layer	Real / controlled boundary	What failure would reveal
Unit tests	Application modules; controlled clocks and external I/O	Parser, branching, budget, validation or query-construction regressions.
Native Connect tests	Actual processor configs inside pinned Connect	Wrong filters, projections, schema/routing or sink mappings.
Integration	Real broker, Connect, SQL and API; controlled external HTTP	Incorrect acknowledgements, replay, migrations and end-to-end payloads.
Mutation probes	Intentional isolated defects	Whether important assertions detect broken behavior.
Live provider/source runs	Actual Wikipedia/Ollama/OpenAI	Feasibility, external compatibility and observed quality limitations.
CI artifacts / audits	Recorded JUnit, coverage, logs and point-in-time advisory data	Reproducibility and reviewable evidence; not proof of all future behavior.

Historical growth, not a count to memorize

Checkpoint	Unit	Integration	Connect	Covered lines / branches
Foundation initial	86	3	-	176 / 32
Foundation maintenance	86	4	-	176 / 32
Ingestion + launchers	99	10	40	176 / 32
Enrichment	214	10	40	451 / 142
Reasoning	333	21	40	829 / 236
Serving	371	25	72	946 / 242
Acceptance	416	29	72	1082 / 276
Demo fix	416	30	72	1082 / 276

Every listed application coverage checkpoint is 100% lines and branches without exclusions. Shell scripts, YAML, infrastructure and model quality require their own evidence; they are not included in that Python percentage. Earlier commands run current code when used at HEAD, so their historical test counts only apply to the recorded commit.

> **Latest recorded gate**
> Local and Linux CI at 71a0680: 416 unit / 30 integration / 72 Connect, 1,082 lines and 276 branches, zero missing/excluded coverage, zero warnings and zero warning annotations.

Source trail
docs/testing.md · docs/evidence/submission-demo-verification.json · docs/evidence/startup-launchers.json · docs/evidence/phase-4-mutations.json

20 / STUDY CHAPTER

Local versus frontier: one contract, explicit choices

Purpose: support different reviewer hardware without hiding data movement or cost.

Dimension	Local Ollama	Hosted frontier
Selected model	qwen2.5:7b	gpt-6.1-sol via OpenAI
Private profile	.env.local	.env.frontier
Compose definition	docker-compose.yml	compose.infrastructure.yml
Inference location	Bundled Ollama container by default	External API; evidence is sent to provider
Startup dependency	Ollama healthy, then model-init completes	Valid hosted settings/key; no Ollama download
Costs / constraints	Local memory, disk, compute and download time	Provider access, network, latency, usage charges
Shared behavior	Same reasoning schemas, prompts, call budgets and validation	Same reasoning schemas, prompts, call budgets and validation

Startup scripts validate the selected private profile, stop the previous stack while retaining data, copy the chosen file over .env, and launch its Compose configuration. Model-only tests read their profile without switching active .env. If startup fails after selection, the selected .env remains for diagnosis and retry; there is no automatic rollback.

For a different bundled local model, keep OLLAMA_MODEL and LLM_MODEL equal: one chooses what is downloaded, the other what is requested. The optional native-host Ollama path has separate networking instructions. It was not the path used in the recorded local runs.

Hosted support is specifically the implemented OpenAI contract, with its fixed base URL. Configurable model names do not mean arbitrary providers, endpoints or models are verified. Another provider requires a deliberate adapter and contract/evaluation tests.

> **Useful explanation**
> There is no silent local-to-hosted fallback. That makes costs, data transfer and provider identity explicit, and makes failures visible instead of quietly changing the experiment.

Source trail
docs/startup.md · src/triage/config.py · src/triage/llm.py · scripts/start.sh · scripts/test-local.sh · scripts/test-frontier.sh

21 / STUDY CHAPTER

What the final real demonstrations showed

Read delivery, semantic assessment and evaluation separately.

Observed measure	Ollama	Frontier
Fixed acceptance cases	9 / 9	9 / 9
Evaluation calls	14	14
Evaluation duration	97.904 s	39.756 s
Live edits persisted	3	3
Live calls	6	4
Live decisions	1 review; 2 insufficient_evidence	1 review; 1 routine; 1 insufficient_evidence
Reason for abstentions	Invalid extraction output	Incomplete source evidence
Processing / sink errors	0 / 0	0 / 0
Storage-consumer lag	0	0
Total model calls	20 local	18 paid

The local run started with fresh Qwen weights; evaluation plus bootstrap took 165.757 seconds. Total evaluation/startup/live durations were 362.216 seconds local and 64.240 seconds frontier. These are single-run observations, not controlled throughput comparisons; the live edits differ and local bootstrap adds work.

The nine fixed cases include two routine edits, five substantive edits and two no-call abstentions. They were frozen before their first Phase 6 run and separate from development tuning. The final demo reuses them, so it is regression evidence, not a new unseen evaluation. Success on nine curated cases does not establish general accuracy on messy Wikipedia diffs.

All three outputs per provider reached storage, including abstentions. The frontier total of 18 calls remained below the declared 40-call ceiling. Currency cost was not measured. No extra paid rerun was used to improve the recorded result.

> **Balanced interpretation**
> Ollama demonstrated safe delivery but successfully assessed only one of three live edits in this run. Frontier assessed two. That is a limitation to investigate, not a reason to relabel abstentions as successful semantic judgments.

Source trail
docs/evaluations/submission-demo.md · docs/evidence/submission-demo-verification.json · docs/evaluations/acceptance.md

22 / STUDY CHAPTER

Docker lifecycle, cleanup and persistence

Purpose: make operational actions understandable and scoped.

Action	Preserves	Removes / changes
clean-docker.sh	Broker data, Postgres rows, model weights and private profiles	This selected project's services/network.
clean-docker.sh --delete-data	Model weights and private profiles	This project's broker and Postgres data.
clean-ollama.sh	All persistent data/weights	Stops the local project, including bundled Ollama.
clean-ollama.sh --delete-models	Broker/database data and private profiles	Only this project's downloaded model weights.
start-local/frontier.sh	Existing persistent volumes	Replaces active .env with the selected private profile.

Destructive cleanup verifies project/logical-volume ownership and rejects external volumes. It does not run global Docker prune or delete native-host Ollama models. Images and build caches may remain; "clean" here is intentionally scoped to the application resources requested.

The executed demo used an isolated project with random loopback ports, empty data and three edits per provider. After frontier completion, default stop preserved all three demo volumes. Starting only sink and api restored the identical three-row API page, with no worker, ingestion or new inference.

Explicit data cleanup retained model weights; explicit model cleanup then removed those weights. Every original volume name remained, private profiles retained their original contents/permissions, and the demo resources were gone. The two proven helper orphan volumes from the pre-fix local run were removed by their recorded names only.

> **One-shot service status**
> topics, db-migrate, model-init and a completed bounded worker can show Exited (0) on a successful run. Healthy long-running containers alone do not prove that an edit reached the API.

Source trail
src/triage/operations.py · docs/operations.md · compose.infrastructure.yml · docs/evidence/submission-demo-verification.json

23 / STUDY CHAPTER

Failure map: where does this input end up?

Purpose: avoid conflating exclusion, abstention and technical failure.

Situation	Destination / behavior	Operator interpretation
Bot / irrelevant / unselected edit	Dropped by ingestion policy	Expected exclusion; not an error or model decision.
Malformed candidate source event	ingest-errors	Inspect source contract/framing; raw payload is hashed.
Missing / oversized / empty valid evidence	results with insufficient_evidence	Safe, inspectable no-call outcome.
Unsupported diff / technical enrichment failure	processing-errors	Investigate the enrichment boundary.
Two invalid model outputs for a stage	results with insufficient_evidence	Semantic output cannot be trusted within budget.
Exhausted model transport retries	processing-errors	Restore provider; deliberate requeue is needed.
Publish or commit failure	Worker exits; input remains pending as appropriate	Restart after fixing broker issue; duplicates possible.
Invalid result envelope / key / partition	sink-errors	No normal SQL row; investigate producer or contract.
Postgres outage	Sink retries; input unacknowledged	Restore DB; replay/lag catch-up, not bad-message quarantine.
API database unavailable	Liveness 200; readiness/data 503	Process alive does not mean dependency ready.

A worker batch can exit 0 after committing processing-error records. Therefore "processed three" is insufficient evidence of three stored analyses. Check each output_topic, both error topics, storage-group lag and the expected result keys.

```
docker compose logs --tail=60 worker connect sink api
docker compose exec -T redpanda rpk group describe triage-v1
docker compose exec -T redpanda rpk group describe triage-store-v1
docker compose exec -T redpanda rpk topic consume processing-errors --offset :end --format
  '%v\n'
```

Read diagnostics; --offset :end reads up to the current end instead of waiting forever

Source trail
src/triage/worker.py · connect/ingest.yaml · connect/sink.yaml · docs/submission-demo.md

24 / STUDY CHAPTER

A clean presentation demo, step by step

Run one operation at a time; preserve reports before rerunning.

```
cd /Users/damien/Development/sourceControl/redpanda
export UV='uvx --from uv==0.13.4 uv'
make test
make demo-success
make demo-failures
```

Start Docker first. Before PR #13 merges, use its review branch for the 30-test integration total.

Expected: all three exit 0. The full gate reports 416 unit, 30 integration and 72 Connect checks with 100% application line/branch coverage. Separately run make demo-test-failure to show the intentionally red assertion; do not present that nonzero exit as an accidental regression.

Configure and run Ollama

Create private profiles from the example files if missing, without overwriting existing ones. In both profiles set WORKER_MAX_ITEMS=3 and INGEST_SAMPLE_ALL=true for a bounded demonstration. Preserve the chosen project/database password. The local profile must download and request the same model.

```
./scripts/clean-docker.sh --delete-data
./scripts/test-local.sh
./scripts/start-local.sh
docker compose logs -f worker
# After worker_stopped reports processed: 3:
docker compose stop connect
```

Data deletion is explicit: save any existing broker/database results first. Weights are retained.

Expected local model test: PASS 9/9 with a saved report. Live worker: three commits, each inspected for output_topic=results, then a clean stop. Follow the API/detail checks on the next page. Press Ctrl+C to stop following logs if necessary.

Reset data, then switch to frontier

```
./scripts/clean-docker.sh --delete-data
./scripts/test-frontier.sh
./scripts/start-frontier.sh
docker compose logs -f worker
# After the three-edit batch finishes:
docker compose stop connect
```

Hosted calls incur charges; maximum 40 per complete bounded attempt, including evaluation.

Repeat the same API checks and save frontier reports separately. Do not launch another batch merely because a model abstained. Use clean-docker.sh at the end to retain data and weights. The full repository walkthrough includes report-saving commands and troubleshooting.

Source trail
docs/submission-demo.md · docs/operations.md · Makefile

25 / STUDY CHAPTER

Prove the end-to-end result on screen

A short evidence checklist is stronger than showing only containers.

```
DEMO_API="http://$(docker compose port api 8000)"
curl --fail "$DEMO_API/health/ready"
curl --fail "$DEMO_API/results?analysis_version=triage-v1&limit=10"
docker compose exec -T redpanda rpk group describe triage-store-v1
```

Readiness, persisted results and sink lag are separate observations.

Choose an actual revision_id from that list. Replace REVISION below with it; do not use synthetic 101 to imply a real live judgment. A fresh batch normally needs a few seconds for asynchronous sink delivery.

```
curl --fail "$DEMO_API/results/enwiki/REVISION?analysis_version=triage-v1"
```

Template command: substitute an observed revision ID.

Field to show	What to explain
analysis.backend / model / prompt_version	Which implementation produced the recommendation.
analysis.decision / explanation / evidence	What was recommended and what quoted text supports it.
enrichment.provenance / source_url	How the exact revision pair was identified.
extraction / steps / outcome	Whether the edit was assessed, reviewed or safely abstained.
attempts / content_truncated	How much work was done and whether evidence was incomplete.
Storage ID / timestamps / version	What is persisted and how replay is logically identified.

In the saved final demo, storage lag was zero but input lag remained nonzero: local 207 and frontier 10 at their snapshots. This is expected when all eligible edits are ingested but the worker deliberately stops at three. Zero storage lag means published results caught up; it does not mean all accepted source edits were processed.

> **Saved evidence location**
> The executed demo's private logs, API pages/details, JUnit and coverage reports are under reports/submission-demo-20261020/ in Damien's checkout. Sanitized summary evidence and synthetic transcripts are committed; routine live content is not automatically public.

Source trail
docs/submission-demo.md · docs/evidence/submission-demo-verification.json · src/triage/api.py

26 / STUDY CHAPTER

The code tour: follow one input, not every file

Use these entry points to explain responsibilities and invariants.

Stop	File / symbol	Question to answer
1	connect/ingest.yaml	Which frames survive, and how is identity preserved?
2	schema.py · EditEvent	Which invalid events cannot cross the boundary?
3	enrichment.py · enrich; diff.py	Where do actual before/after text and provenance come from?
4	prompts.py; reasoning.py · reason	What does each model stage receive, and when does code branch?
5	reasoning.py · parse_output / validate_grounding	How are malformed output and invented quotes rejected?
6	llm.py · complete	Who selects the provider and owns each individual request?
7	worker.py · process / run_worker	How do results and technical failures route?
8	broker.py · publish / commit	Where is the output-ack-before-input-commit invariant?
9	connect/sink.yaml; db/002-results.sql	What makes replay safe at the SQL boundary?
10	storage.py; api.py	How are stored results validated and paginated?
11	evaluation.py; operations.py; scripts/	How do evaluation, mode switching and cleanup remain bounded?

All Python files above are under src/triage/. Worker and API share one package/container image but run as separate processes. The design intentionally avoids an agent framework, ORM, migration framework and frontend build. That keeps the behavior traceable in the application and Connect configuration.

Walkthrough habit

For each function, name its input contract, allowed output, side effect and failure behavior. For example: publish receives bytes and a topic/key; it returns only after delivery acknowledgement, otherwise raises a sanitized error. It does not commit the input offset. Commit is a separate, ordered operation.

> **Do not memorize line numbers**
> Use symbols and source links in this pack. Line numbers can move; the responsibility and invariant should remain clear.

Source trail
docs/handover.md · src/triage/worker.py · src/triage/broker.py · src/triage/reasoning.py

27 / STUDY CHAPTER

Prompts, responses and evidence provenance

Two audit trails serve different purposes.

Development conversation

The repository records the user's visible requests, clarification answers and the assistant's visible replies in ordered session Markdown. It preserves wording, identifies the phase and links review/evidence where known. It does not claim to capture other chats automatically or record hidden reasoning, system instructions or credentials.

Representative request	Resulting engineering response
"Wikipedia edit triage"	Choose a public source and fetch revision diffs, not just comments.
"Python, keep it as simple as possible"	One small typed package, direct control flow, no agent framework.
"choose a local model or a frontier model"	Explicit adapters and private profiles with documented compatibility.
"I will review and merge each phase"	Dependent phase branches and PRs; user controls integration.
"run the full demo ... save the results"	Bounded live runs, saved successes/abstentions, cleanup regression and PR #13.

Runtime model evidence

Prompts are versioned source code. Curated evaluations save each visible system/user prompt, response, stage, model and outcome, then publish sanitized synthetic transcripts. Normal worker logs retain bounded metadata; the API retains structured evidence and stage outcomes, not every raw provider response.

The acceptance evidence records frozen-input hashes, model identities, call counts, timing, output decisions, lag and exact tested commits. Failed development observations were retained: prompt v1 scored 5/6, v2 6/6 on the development set. That improvement is tuning evidence, not a held-out score.

> **How to discuss AI assistance**
> Explain the code you can defend and the checks that caught mistakes. The audit establishes what was requested and observed; it does not substitute for your understanding or your independent README judgments.

Source trail
docs/ai-log/README.md · docs/ai-log/sessions/2026-10-20-executed-demo.md · docs/evaluations/development.md · src/triage/prompts.py

28 / STUDY CHAPTER

Prioritized improvement roadmap

Proposed work: tie each change to a failure mode and a measurable acceptance test.

Priority / area	Why the current design needs it	Evidence required before calling it done
P1 · Model reliability	Small-model invalid-output abstentions on real diffs; 20 source hunks versus 8 extracted changes	Label realistic edits independently; measure decision precision/recall, abstention rate, grounding and per-item call cost. Preserve an untouched holdout.
P1 · Operational visibility	Logs/lag exist; there is no integrated SLO/alerting	Dashboards for input/result lag, source failures, abstentions, latency and provider errors; exercise actionable alerts.
P1 · Replay operations	Error topics are not automatic scheduled retries	Audited requeue tooling with scoped IDs, retry budgets and duplicate-safe verification.
P1 · Security / data durability	Local roles, no auth/TLS or backup/restore workflow	Separate roles, secret management, authentication, transport protection and a measured restore drill.
P2 · Source continuity	No durable source cursor; disconnects can lose edits	Checkpoint/backfill design with overlap deduplication and a controlled disconnect/recovery test.
P2 · Capacity / concurrency	Single partition and sequential inference; selection is not a rate limit	Load tests, backlog SLO and bounded concurrency; redesign persistence ordering before repartitioning.
P2 · Evidence breadth	No citation retrieval or full-article context	Bounded allowlisted retrieval with attribution, privacy/cost limits and adversarial tests.
P3 · Review workflow	API is inspectable but no reviewer decisions/UI	Human feedback capture, access controls and evaluation of whether reviewers save time.

These priorities are suggested engineering sequencing, not implemented features or the candidate's personal tradeoff essay. Choose actual priorities with a customer's workload, failure cost, privacy requirements and deployment constraints.

First experiment to consider
Replay a fixed, human-labelled sample of real diffs through both providers, including large multi-hunk edits, and inspect why extraction fails. Improve the evidence/model contract before simply increasing retry counts or weakening quote validation.

Source trail
docs/handover.md · docs/evaluations/submission-demo.md · src/triage/reasoning.py · docs/security-review.md

29 / STUDY CHAPTER

What must change for a real customer deployment

Quality practices are present; deployment guarantees remain bounded.

Durability and scaling

The broker is single-node with replication factor one. There is one partition per topic, no worker autoscaling or automatic restart supervision, and no demonstrated recovery objective. Add redundancy, capacity/retention planning and backup/restore tests according to the actual deployment. Kafka retention determines whether replay is still possible.

Repartitioning is a contract change here: source offsets are only ordered within one partition, while the SQL guard assumes partition zero and stable history. A production redesign could use a partition-aware checkpoint and a separate analysis-run identity. It must explicitly decide how multiple models, prompt versions and reanalyses coexist instead of silently overwriting one triage-v1 row.

Security and privacy

The API and diagnostics bind to loopback, broker/database ports are internal, and app containers are non-root/read-only with dropped capabilities. Those reduce local exposure. They do not provide authentication, TLS, tenant isolation or separate read/write/migration roles. Hosted mode sends selected article evidence to a provider; a customer-data version needs an explicit data-handling design.

Recorded security evidence, not a current certification

The maintenance Python audit found no known advisories in 36 installed packages at that point in time. The hardened ARM64 application image had zero critical and zero Python high/critical findings, but 44 high package/advisory pairs across eight Debian advisories remained. The later dependency update did not repeat the OS scan. Broader infrastructure/architecture scanning and vendor updates remain future work.

> **A precise claim**
> The project is a small, tested local reference implementation with explicit failure handling. It is not highly available, lossless from the public source, generally accurate as a classifier, or vulnerability-free.

Source trail
docs/security-review.md · docs/handover.md · docs/evidence/actions-dependency-audit.json · docs/evidence/phase-6-container-audit.json

30 / STUDY CHAPTER

Questions you should be able to answer

Use the answer cues to check understanding, then explain in your own words.

Question	Answer cue
Why does this need an LLM?	The task compares changed meaning/support in real text. Rules still handle eligibility, identity, bounds and safety.
Why use Kafka/Redpanda at all?	Decouple stream, inference and storage; retain/replay work; inspect offsets and lag. The broker is Redpanda, speaking Kafka protocol.
Is Connect doing the reasoning?	No. Connect owns ingestion/sink plumbing; Python owns extraction, assessment, branching and repairs.
Why two model steps?	Extraction creates grounded changes; assessment checks meaning against original hunks. It costs extra calls and does not guarantee correctness.
What happens on a malformed response?	Strict parsing/grounding, bounded repair, then abstention; transport failure is a different technical route.
Does 0.9 confidence mean 90% accuracy?	No. It is uncalibrated model output used as a routing signal.
Why are duplicates possible?	Output may be acknowledged before the input commit succeeds. Restart can repeat inference; SQL handles logical duplication.
Can I add partitions now?	Not without redesigning the sink's single-partition offset guard and testing ordering/replay.
Why can the API be healthy with missing results?	Readiness checks storage availability, not completion of upstream processing. Check keys, errors and lag.
Does 100% coverage mean no bugs?	No. It records executed application lines/branches under tests, not model correctness or all possible production behavior.
Why did both models pass 9/9 but live outputs abstain?	Small curated cases differ from large messy diffs. Regression success does not generalize automatically.
What is the cheapest replay?	Replay results into storage with a new sink group; replaying edits through the worker invokes inference again.

Source trail
docs/handover.md · docs/testing.md · docs/evaluations/acceptance.md · src/triage/reasoning.py

31 / STUDY CHAPTER

A 60-minute walkthrough plan

Allocate time to evidence and questions, not only architecture slides.

Time	Show	Explain / invite discussion
0-5 min	Problem, scope and one stored result	What the result means; what the system does not claim.
5-12 min	Architecture + phase dependency map	Why transport, reasoning and serving are separated.
12-22 min	One event through ingestion and enrichment	Filters, keys, provenance, limits and actual before/after text.
22-34 min	Reasoning function + worked example	Extraction, assessment, conflict, repair and shared budgets.
34-43 min	Replay/failure demo + sink/API	Ack/commit order, duplicate handling, DB recovery and health semantics.
43-50 min	Local/frontier recorded results	Same contract, explicit costs, observed abstentions and quality limits.
50-56 min	Extension diff or proposed change path	What changes where, how it is tested and what might break.
56-60 min	Open questions + improvement priorities	State uncertainty precisely and point to evidence.

The HTML companion includes a presentation mode for this sequence and a step-through of the event lifecycle. Its interactive traces are explanatory simulations using saved/controlled examples; they do not start Docker, fetch Wikipedia, spend model credits or submit any external action.

Rehearsal checklist

- Explain the causal-claim example without reading the prompt.
- Trace one malformed response and one crash after publication.
- Find the exact quote validator, call budget and offset commit in code.
- Show one complete API detail and distinguish its summary from source evidence.
- Explain one actual defect discovered by a test and how the test proved the fix.
- Describe an improvement with a measurable acceptance criterion.

> **Independent preparation**
> Write your own Tradeoffs, Surprises/production failure modes and Why this matters sections. Use questions, observations and source links here as study material; do not paste this guide as the independently authored submission.

Source trail
docs/assignment/fde-take-home-exercise.md · docs/submission-demo.md · docs/handover.md

32 / STUDY CHAPTER

Preparing for the pre-walkthrough extension

Change the smallest boundary that actually owns the new requirement.

The assignment may supply a small change to the reasoning service shortly before the walkthrough. No specific extension has been supplied yet. The examples below are rehearsal exercises, not commitments or implemented features.

Hypothetical change	Likely impact	Essential checks
Add a confidence tier	Reasoning/analysis contract; exported schemas; sink/API compatibility; version decision	Boundary values, abstentions, old/new envelope compatibility and end-to-end field persistence.
Route urgent edits to a new topic	Application routing after reasoning; topic initializer; downstream consumer design	Output acknowledgement still precedes input commit; failure leaves input pending; correct routing and idempotency.
Add a known-bot skip rule	Confirm requirement scope: source filter or worker policy; do not infer from arbitrary instructions in content	Targeted negative/positive examples, no unnecessary model calls, explicit outcome semantics.
Thread a new source field end to end	Connect projection; EditEvent; schemas; enrichment/result/API; storage expectations	Exact fixtures at every boundary, schema evolution and no accidental raw metadata leakage.

A repeatable change process

- Restate the input, new observable output and compatibility decision.
- Locate the owning layer and the contracts crossing it.
- Create a focused test that fails for the missing behavior.
- Implement the smallest change and run targeted checks, then the cumulative gate.
- Inspect one complete input-to-output record; update phase evidence and the visible prompt log.
- Open a reviewable diff; explain remaining limits and migration/replay implications.

> **Avoid a common trap**
> Adding a field is not complete because it appears in one Python class. Strict schemas, Connect validation, saved fixtures and the API may all reject or omit it until the contract is changed end to end.

Source trail
docs/assignment/fde-take-home-exercise.md · src/triage/schema.py · schemas/reasoned-edit.v1.json · connect/sink.yaml

33 / STUDY CHAPTER

Glossary and quick distinctions

Use these terms precisely during the walkthrough.

Term	Meaning in this project
SSE	Server-sent events: the public stream framing consumed by Connect.
Bloblang	Connect's mapping language for filtering, projecting and routing messages.
Topic / partition	A named Kafka-compatible log and its ordered shard; this deployment uses one partition per topic.
Key	Stable identity: enwiki:revision for input; plus analysis version for results. A key alone does not prevent duplicate records.
Consumer group / offset	A consumer identity and position in a partition. A committed offset points to the next record to process.
Acknowledgement	Confirmation that the broker accepted an output; distinct from committing the input position.
At least once	Uncommitted work may replay, so duplicate processing/output can occur.
Idempotent storage	Repeating an analysis identity does not create duplicate logical SQL rows.
Enrichment / provenance	Fetched before/after content, plus evidence of which revision pair and endpoint produced it.
Grounding	Validating exact source excerpts; does not itself prove the model's interpretation is correct.
Abstention	An explicit insufficient_evidence result; different from a technical processing error.
Confidence	Uncalibrated model signal used for branching; not measured likelihood of truth.
Fixture / live check	Controlled reproducible input versus an observation of real external behavior.
Liveness / readiness	The process responds versus its required storage is usable; neither proves all work is complete.
Model / prompt / analysis version	Provider identity, prompt-template identity and logical storage namespace; separate concepts.
Coverage	Executed application lines/branches under unit tests; not source completeness, quality or uptime.

Keep one final distinction in mind: the two Connect services run the same product with different configurations. Redpanda is the Kafka-compatible broker. Ollama is the local inference server. Docker Compose runs the local topology. None of those is an agent framework.

Source trail
docs/handover.md · src/triage/broker.py · src/triage/schema.py

34 / STUDY CHAPTER

Source map and evidence register

Prefer exact code and retained observations over recollection.

All repository links in this pack are pinned to snapshot 859b166 unless they point to a PR, commit or CI run. That prevents the guide's claims drifting when main changes. Older phase documents sometimes contain historical "open PR" wording; the phase map in this pack uses the actual review status checked at preparation time.

Study question	Primary repository evidence
What was asked?	docs/assignment/fde-take-home-exercise.md; docs/implementation-plan.md
What changed by phase?	docs/phases/00-plan.md through 06-acceptance.md; docs/progress.md
What is actually running?	compose.infrastructure.yml; docker-compose.yml; Dockerfile; uv.lock
How does an event move?	connect/ingest.yaml; src/triage/worker.py; connect/sink.yaml
What does the model see?	src/triage/prompts.py; docs/evidence/submission-model-*.json
What did final runs prove?	docs/evaluations/submission-demo.md; docs/evidence/submission-demo-verification.json
How do I reproduce checks?	Makefile; docs/operations.md; docs/submission-demo.md
Where are limits disclosed?	docs/handover.md; docs/security-review.md; docs/evaluations/acceptance.md
Where is the development audit?	docs/ai-log/README.md and its linked session files.

Key review links: PR #7 enrichment; #8 reasoning; #9 persistence/API; #11 operational acceptance and CI; #13 executed demo and helper cleanup. Latest implementation verification used 71a0680 (implementation ce40ca6), with Linux CI run 39494682046. Snapshot 859b166 adds documentation/evidence closeout only.

The original assignment's preserved SHA-256 is 89a4840092423818c59a81f4042f9007d36e90fe7da7bc8111a4d96f1662e40a. This study pack does not alter the assignment, runtime code, private profiles or personal README sections. Generated-artifact QA is separate from the application's recorded unit coverage.

> **How to use the links**
> Each section ends with direct source links. The editable guide includes the complete narrative; the HTML adds local interactions and presentation mode. Use the repository's commands for real execution, and the saved evidence to discuss what actually happened.

Source trail
docs/progress.md · docs/evidence/submission-demo-verification.json · docs/assignment/README.md